ML ENGINEERING PROJECT BLUEPRINT — CONDENSED EDITION

Multimodal Image & Text Semantic Search Engine

A production-oriented retrieval system built on CLIP/SigLIP embeddings, PostgreSQL + pgvector, and FastAPI — from a Flickr30k prototype to an e-commerce-ready architecture. This is a 10-page condensed summary of the full 37-section blueprint.

PREPARED FOR	Soufian — Data Science & AI Engineering
DOCUMENT TYPE	Condensed project plan & portfolio blueprint
PRIMARY DATASET	Flickr30k (prototype) → ABO (production path)
CORE STACK	PyTorch · Hugging Face · PostgreSQL/pgvector · FastAPI · Docker
SCOPE	Data engineering → Modeling → API → MLOps → Deployment

MVP-first design

Evaluation-driven

Student-budget deployment

01 Executive Summary & Use Case

This project is a **multimodal semantic search engine**: users search a collection of images by text, by a reference image, or both, and receive ranked results with similarity scores. Images and text are projected into a shared vector space using a pretrained CLIP/SigLIP encoder, stored in PostgreSQL + pgvector, retrieved via approximate nearest-neighbor search, and served through a tested, monitored, deployed FastAPI service. The engineering — not the model call — is what separates this from a notebook demo: ingestion validation, persisted/versioned embeddings, indexed search, formal evaluation, a real API, tests, Docker, CI/CD, and monitoring.

Why pretrained models: CLIP/SigLIP were trained on hundreds of millions of pairs — infeasible to reproduce as a student. The value is in correctly integrating, evaluating, and productionizing a strong pretrained encoder, then improving it surgically only where evaluation shows it's worth the cost.

Real-world use cases

Use case	Needs from this architecture	Examples
Visual product search	Image→image retrieval, fast ANN	Pinterest Lens, Google Lens shopping
Text-based product discovery	Text→image retrieval + attribute filters	"red midi dress, long sleeves"
Digital asset management	Natural-language search over unlabeled libraries	Marketing/media asset libraries
Recommendation ("more like this")	Retrieval as a candidate generator	"Similar items" rails

Flickr30k de-risks the architecture (small, clean captions, no licensing complexity); every component is dataset-agnostic and migrates to a product catalog like ABO with minimal change (Section 09).

02 Requirements

Functional	Priority
Text → Image search	MVP
Image → Image search	MVP
Image + Text fused search	RECOMMENDED
Ranked results w/ image_id, caption, score, rank	MVP
POST /search/text, /search/image, /search/multimodal, GET /health, /model/info	MVP

Non-functional	Target
Latency (text)	P50 < 300ms, P95 < 800ms CPU
Latency (image)	P50 < 500ms, P95 < 1200ms
Scalability	Sub-second up to ~50k–100k vectors, single small VM
Reproducibility / Testability	Deterministic embeddings; >70% coverage on core logic
Security / Observability	Upload validation, no secrets in repo; structured per-request logs

03 Dataset, Data Engineering & EDA

Flickr30k	Value
Images / Captions	≈31,783 images · 5 captions each (≈158,900 total)
Split	Karpathy split: ≈29k train / 1k val / 1k test
License	Non-commercial research license — state explicitly in README
Limitations	Generic scenes, no structured attributes, some demographic bias — exactly what ABO migration addresses (Section 09)

Download → Integrity check → Corruption & duplicate detection → Missing-image check

↓

Caption cleaning → Metadata validation → Deterministic train/val/test split

↓

Validated dataset + data-quality report (data/metadata/)

EDA focus areas

Image dimensions/aspect ratios (resize distortion risk) · caption length & vocabulary (tokenizer truncation, domain breadth) · missing data · exact & near-duplicate images/captions · image-caption relationship spot checks · file-size distribution.

Preprocessing

Image: force RGB, resize/normalize to the encoder's exact documented recipe (mismatched normalization silently degrades quality). **Text:** minimal — whitespace/Unicode normalization + length cap only.

> **WHY AGGRESSIVE NLP PREPROCESSING HURTS CLIP/SIGLIP**
>
> Classical NLP steps (stopword removal, stemming) were built for bag-of-words models. CLIP/SigLIP text encoders are transformers trained on natural captions and rely on word order and function words for context — stripping them shifts queries away from the training distribution and typically *reduces* retrieval quality. Preprocess for validity, not classical "cleanliness."

04 Model Strategy & Embedding Pipeline

Aspect	CLIP	SigLIP
Objective	Contrastive softmax (InfoNCE-style)	Sigmoid pairwise loss, no large-batch requirement
Recommendation	Start with CLIP (best documented) as baseline; add SigLIP as a comparison experiment (Section 06).	

Why not train from scratch: requires hundreds of millions of pairs and thousands of GPU-hours — infeasible for a student. Fine-tune only after evaluation shows a measured domain gap (Section 06) — ADVANCED for Flickr30k, worthwhile once moved to a specialized domain like ABO.

Image → preprocess → vision encoder → L2-normalize → store (pgvector, tagged w/ model+version)

Caption/query → tokenize → text encoder → L2-normalize → store or use transiently

Batch offline embedding (10–50x throughput vs. one-at-a-time). GPU only needed for the one-time full-dataset embed (free-tier Colab/Kaggle is enough); live query embedding runs fine on CPU. Every embedding tagged with model+version+config hash to avoid recomputation.

05 Database Design

Table	Purpose
images	image_id, path/url, dims, file_hash, split, ingested_at
captions	caption_id, image_id (FK), text — generalizes to titles/descriptions for ABO
image_embeddings / text_embeddings	embedding_id, ref FK, model_name+version, vector, created_at
models	registry of served encoder versions, feeds /model/info
search_logs / evaluation_runs RECOMMENDED	monitoring & tracked evaluation history

pgvector: native vector(N) column; cosine distance ≡ dot product once vectors are L2-normalized. No index = brute-force scan (fine to ~10–20k rows). **HNSW** = default choice at Flickr30k scale (best latency/recall). **IVFFlat** = cheaper to build, needs a representative sample, degrades faster under updates.

06 Retrieval, Multimodal Fusion & Evaluation

Term	Definition here
Retrieval	Cheap Top-K narrowing via the vector index — optimized for speed/recall
Ranking	Precise ordering of the small candidate set, can use a costlier model
Recommendation	Broader task layering in user history/business rules — out of MVP scope

Multimodal fusion approaches

Approach	Tradeoff
A. Weighted embedding combination MVP	alpha·image + (1-alpha)·text, one index lookup — simple, fast, recommended default
B. Separate retrieval + score fusion RECOMMENDED	Two lookups, more robust when one modality dominates
C. Dedicated fusion model ADVANCED	Needs labeled multimodal query data to train

Evaluation set & metrics

Build queries from held-out test-split captions; ground truth = the caption's source image. Never let test-set observations influence preprocessing/tuning decisions (leakage risk); dedupe across splits to avoid trivial near-duplicate matches.

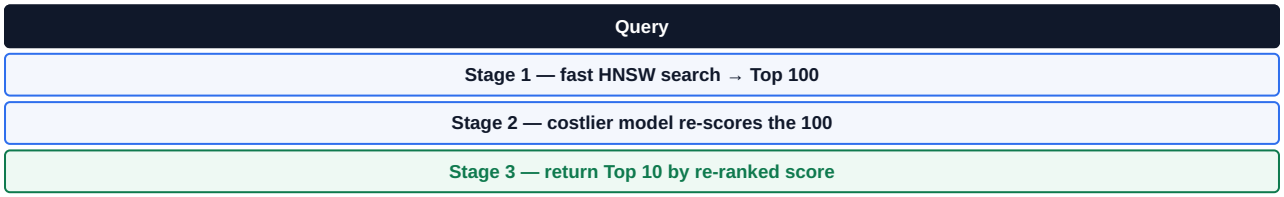
Metric	Meaning	MVP target (CLIP ViT-B/32 baseline)
Recall@1	True positive is rank #1	≈25–35%
Recall@5 / @10	True positive in top 5 / 10	≈50–65% / ≈65–78%
MRR	Mean of 1/rank of first correct result	rewards early correct hits
NDCG RECOMMENDED	Graded relevance, ideal-ranking normalized	most useful once relevance is graded (re-ranking, ABO attributes)

07 Experiments, Improvement & Re-Ranking

#	Experiment	Priority
1	Random baseline (floor)	MVP
2	CLIP ViT-B/32 baseline	MVP
3	SigLIP comparison	RECOMMENDED
4–5	Similarity-metric & normalization ablations	RECOMMENDED
6	Re-ranking on top of best retrieval	ADVANCED

Log every run's params/metrics in MLflow — never fill a results table in by hand.

Two-stage re-ranking ADVANCED



Decouples ranking quality from index size: the expensive model only ever scores a fixed small candidate set. Stage 2 options in increasing cost: bigger encoder re-score → metadata-fused scoring → cross-encoder (query+candidate jointly).

Model improvement — when it's worth it

Fine-tune only once evaluation shows a measured, systematic gap the pretrained model can't close zero-shot. Hard-negative mining and contrastive/triplet loss sharpen the boundary further, but require training infrastructure and labeled pairs — reserve for the ABO/product-domain phase.

08 API, Frontend, Repo Structure & Testing

Endpoint	Behavior
POST /search/text	query, top_k → ranked results (image_id, url, caption, score, rank)
POST /search/image	multipart upload → same result shape; 413/415/422 on bad uploads
POST /search/multimodal	image + text + alpha → fused results
GET /health / /model/info	DB+model status; served model_name/version/dim

Repo: data/ · notebooks/ (never imported by prod code) · src/ (installable package, single source of logic) · tests/ · configs/ · scripts/ · models/ · docker/ · frontend/ · docs/ · .github/workflows/ · Dockerfile · docker-compose.yml · README.md.

Frontend (kept simple): text box, image upload, multimodal alpha slider, result grid with score bars, loading skeletons, inline error messages.

Testing strategy

Type	Example
Unit	Normalization → unit-length vectors; fusion at alpha=0/1
Data	Corrupted-image fixture quarantined; deterministic split given a seed
Embedding	Same image twice → identical vector; matching pair scores > random pair
Retrieval	Query = stored embedding → returns itself as rank 1
Integration	Full query→embed→pgvector→enriched-result path vs. a seeded test DB
API	Valid query → 200 + correct schema; oversized upload → 413; DB outage → /health 503

Target >70% coverage on src/ business logic, gated in CI.

09 MLOps, CI/CD, Monitoring, Security & Deployment

Area	What to do
MLflow	Log params (model, index type, K, alpha), metrics (Recall@K/MRR/NDCG, latency), artifacts (eval report), model versions
Versioning	Pin checkpoint revision (never "latest"); log dataset manifest hash + git commit per run
CI/CD	lint+test → integration test (throwaway Postgres/pgvector container) → Docker build+smoke test → deploy on merge, gated end-to-end
Monitoring	P50/P95/P99 latency, error rate, throughput, embedding latency, model_version — Prometheus + a small Grafana dashboard is enough for MVP
Security	Upload type/size validation, safe decode of untrusted files, per-IP rate limiting, env-var secrets, least-privilege DB user, restricted CORS

Deployment (student-budget)

Browser → Frontend (static host) → FastAPI (Dockerized, low-cost/free VM) → PostgreSQL+pgvector (same VM, Docker Compose) → image volume

Serve on CPU (single-query latency is fine at this scale, Section 04); GPU only used transiently for the one-time offline embedding pass. Oracle Cloud Always Free ARM (or Fly.io/Render free tier) + self-hosted Postgres keeps cost near zero.

10 Milestones & Week-by-Week Plan (8 weeks, part-time)

#	Milestone	Acceptance criteria
1	Dataset	Zero corrupted images remain; reproducible split; EDA report complete
2	Database	Schema applied, FK constraints enforced, smoke-test round-trip passes
3	Embedding pipeline	One embedding per (image, model_version); re-run doesn't duplicate
4	Baseline retrieval	Hand-picked queries return visibly relevant results
5	Evaluation	CLIP clears random baseline by a wide margin; results reproducible
6	Model improvement	≥2 additional experiments logged with a clear conclusion
7	API	All MVP endpoints schema-valid; correct error codes on bad input
8	Frontend	All search modes work end-to-end from the browser
9	Docker	Fresh clone + docker compose up reaches a working /health, no manual steps
10	CI/CD	Broken test blocks merge; passing PR builds & smoke-tests the image
11	Monitoring	Induced error visible on the dashboard within the scrape interval
12	Deployment	All prior acceptance criteria hold true against the live deployment

Week	Focus
1	Data download, integrity/corruption/duplicate checks, reproducible split
2	EDA report; PostgreSQL+pgvector stood up; schema applied
3	Batched embedding pipeline run end-to-end; determinism verified
4	HNSW index built; retrieval implemented; evaluation harness + first baselines logged to MLflow
5	FastAPI endpoints, validation, logging, error handling; API tests
6	Frontend built; Dockerfile + docker-compose wiring the full stack
7	GitHub Actions CI/CD; structured logging + Prometheus/Grafana dashboard
8	VM provisioning, deploy step wired, public verification, README/demo polish

11 Portfolio, Interview Prep & Final Checklist

README structure: Title → one-liner → demo link/GIF → architecture diagram → results table (Recall@1/5/10, MRR, P50/P95 latency across ≥2 configs) → how to run locally (Docker Compose) → design decisions & tradeoffs → roadmap.

> **EXAMPLE CV BULLET**
>
> "Built and deployed a multimodal semantic search engine (CLIP, PostgreSQL/pgvector, FastAPI, Docker) over 31k images, achieving Recall@5 of [X]% and P95 latency under 800ms; established reproducible evaluation, MLflow tracking, and CI/CD, with a validated migration path to a product-search domain."

Interview questions to be ready for

Question	Core answer
What is CLIP?	Dual encoders trained contrastively into a shared embedding space for zero-shot cross-modal similarity
Why normalize embeddings?	L2-normalization makes cosine similarity = dot product — cheaper to index without changing ranking
Why a vector DB over brute-force NumPy?	Indexed ANN search scales sub-linearly, plus persistence, transactions, and joins against metadata
How does HNSW work?	Multi-layer graph; search descends from a sparse top layer into denser layers toward the neighborhood
Recall@K vs. MRR?	Recall@K is binary per query within K; MRR is continuous and rewards ranking correct results earlier
When to fine-tune?	Only after evaluation shows a measured, systematic domain gap zero-shot can't close
Why retrieve-then-rerank?	Cheap stage narrows the index; expensive stage only scores a small fixed candidate set — decouples cost from index size

Final checklist

- [ ] Dataset validated & split
- [ ] EDA report done
- [ ] Preprocessing implemented
- [ ] PostgreSQL + pgvector running
- [ ] HNSW index built
- [ ] Embeddings generated & stored
- [ ] Text / image / multimodal search working
- [ ] Evaluation set + Recall@K/MRR
- [ ] Baseline + comparison experiments logged
- [ ] Fine-tuning decision documented
- [ ] FastAPI service + validation
- [ ] Frontend connected
- [ ] Unit/integration/API tests passing
- [ ] Docker Compose verified clean-clone
- [ ] MLflow tracking runs
- [ ] CI/CD gating merges
- [ ] Monitoring dashboard live
- [ ] Security checklist applied
- [ ] Deployed publicly
- [ ] README + architecture diagram
- [ ] CV bullet + interview prep reviewed